

# verilog简介

1. VerilogHDL是一种用于**数字逻辑电路**设计的硬件描述语言 (HardwareDescriptionLanguage), 用来进行数字电路的仿真验证、时序分析、逻辑综合。
2. 结构类似cpp

## 基础知识

### 关键字

定义:事先定义好的确认符, 用来组织语言结构; 或者用于定义VerilogHDL提供的门元件 (如 and, not, or, buf)

| 关键字      |              |             |           |           |          |          |
|----------|--------------|-------------|-----------|-----------|----------|----------|
| and      | edge         | function    | negedge   | rcmos     | strength | tri0     |
| always   | else         | highz0      | nor not   | real      | strong0  | tri1     |
| assign   | end          | highz1      | notif0    | realtime  | strong1  | vectored |
| begin    | endcase      | if          | notif1    | reg       | supply0  | wait     |
| buf      | endfunction  | ifnone      | nmos      | release   | supply1  | wand     |
| bufif0   | endprimitive | initial     | or        | repeat    | table    | weak0    |
| bufif1   | endmodule    | inout       | output    | rnmos     | task     | weak1    |
| case     | endspecify   | input       | parameter | rpmos     | tran     | while    |
| casex    | endtable     | integer     | pmos      | rtran     | tranif0  | wire     |
| casez    | endtask      | join        | posedge   | rtranif0  | tranif1  | wor      |
| cmos     | event        | large       | primitive | rtranif1  | time     | xnor     |
| deassign | for          | macromodule | pulldown  | scalared  | tri      | xor      |
| default  | force        | medium      | pullup    | small     | triand   |          |
| defparam | forever      | module      | pull0     | specify   | trior    |          |
| disable  | fork         | nand        | pull1     | specparam | trireg   |          |

### 标识符

定义:

用VerilogHDL语言描述的“东西”的**名字**, 类似cpp语言里面的**自定义变量名**。

命名规则:

1. 标识符可由字母、数字、下划线和符号构成; 但第一个字符必须是字母或下划线, 不能是数字或\$符号。
2. VerilogHDL中变量名要区分大小写。
3. 标识符与关键字不能同名。

### 数据类型

## 基本数据类型：

1. wire：表示物理连线，用于 assign 语句。对应硬件：电路中的一根导线。
2. reg：表示存储单元或过程赋值中的变量。能够保持住上一次赋予的值，直到下一次赋值。对应硬件：触发器or组合逻辑中的临时变量
3. integer：32位有符号整数。
4. parameter：符号常量，可定义延迟、位宽等。parameter本身是一种**常量**，也是一种**数据类型**。
  - **作为数据类型**：它告诉编译器，这个标识符（如 SIZE）是一个常量，而不是变量。
  - **作为常量**：它代表了一个固定的值（如 SIZE 的值是16）

## 基本数据类型vs常量：

- **基本数据类型**：定义了一个**容器**的性质（比如它是一个电线、一个寄存器、一个整数变量）。
- **常量**：指的是存放在容器里的**固定不变的内容**。

## 运算符

### 逻辑运算符：

- && 与
  - || 或
  - ! 非
- 非零的操作数被认为是真(1'b1)；零被认为是假(1'b0)

### 位运算：

- ~ 按位取反
  - & 按位与
  - | 按位或
  - ^ 按位异或
- 对操作数的每一位**进行独立运算**。 **结果位宽**与操作数位宽相同，位宽不同时，**高位补0**再运算

## 语句

### 赋值语句

#### 1.连续赋值

assign语句，用于对wire型变量赋值

```
assign c=a&b;    //a、b、c均为wire型变量
```

## 2.过程赋值

用于对reg型变量赋值，有两种方式：

1. 非阻塞：非阻塞赋值在块结束时才完成赋值操作；块内的多条赋值语句在块结束时同时赋值；硬件有对应的电路

```
b<=a
```

2. 阻塞：阻塞赋值在该语句结束时就完成赋值操作；完成该赋值语句后才能执行下一句的操作；硬件没有对应的电路

```
b=a
```

建议在可综合风格的模块中使用**非阻塞赋值**

## 块语句

用于将多条语句组合成一个整体

### 1.串行块 `begin ... end`

- 行为：块内的语句按书写顺序**顺序执行**。
- 延迟：语句中的延迟时间（如 `#10`）是相对于前一条语句完成时的**相对时间**

```
begin
    b<=a;
    c<=b;
end
//结束时b为a，c为原来的b，同时发生
```

```
begin
    b=a;
    c=b;
end
//顺序执行，b为a，c为b即c为a
```

### 2.并行块 `fork ... join`

- 行为：块内的语句**同时开始执行**。
- 延迟：语句中的延迟时间是相对于进入块时的**绝对时间**。

## 结构说明语句

### 1.always

行为：**不断重复执行**其中的语句，是描述**时序逻辑**和**组合逻辑**的核心。

格式：always<时序控制><语句>

- **电平敏感** ( always @(a or b or c) ): 通常用于描述**组合逻辑**。必须列出所有输入信号。
- **边沿敏感** ( always @(posedge clk) ): 通常用于描述**时序逻辑**（寄存器、计数器、状态机）  
模板：
  - **时序逻辑模板**: always @(posedge clk or negedge reset\_n)
  - **组合逻辑模板**: always @(\*) （推荐，自动生成敏感列表）

```
always @ posedge min_clk or negedge reset)
begin
    if (reset)
        min<=0;
    else if (min=8'h59) //当reset无效且min=8'h59时；这里不能写if
        begin
            min<=0;
            h_clk<=1;
        end
    end
end
```

## 2.initial

```
initial
begin
    inputs='b000000;
    #10 inputs='b011001
    ...
end
```

## 条件语句

必须放在 always 块内!!!

### 1. if-else 语句

当if与else的数目不一样时，最好用“begin\_end”语句将单独的if语句括起来  
else if方式最后不一定要带else

### 2. case 语句

```
always@(sel[1:0]or a or b)
case(sel[1:0])
    2'b00:q<=a;
    2'b11:q<=b;
```

```
default: q<='b0;  
endcase
```

## 循环语句

### 1.for

当执行语句有多条时,可用begin\_end语句将其括起来。类似cpp语言的中括号for(int i=0 i<n ; i++){}

### 2.while

### 3.repeat

### 4.forever